

Lab0:GitLab 实验报告

一、实验目的

- 1.掌握 Git 的基础操作：仓库克隆、文件修改、暂存、提交、推送。
- 2.理解 Git 暂存区与提交的工作机制，理解 `git branch` 分支命令的用法。
- 3.学习多分支开发流程，制造、识别并手动解决代码合并冲突。
- 4.了解 Git 团队协作的规范，理解学习版本控制工具的意义。

二、实验环境

- 1.操作系统：Windows + WSL2 (Ubuntu)
- 2.编辑器：VS Code
- 3.版本控制工具：Git
- 4.远程仓库平台：GitHub

三、实验步骤

1.任务 2：修改 main.c 并完成一次 commit

从模板仓库克隆个人仓库到本地 WSL 环境。

使用 VS Code 打开仓库，找到 main.c 文件，修改 TODO 对应的 printf 输出语句。

在 WSL 终端执行命令，保存改动并提交、推送到远程：

```
1 git add main.c
2 git commit -m "lab0: 修改 main.c, 完成输出语句"
3 git push
```

打开 GitHub 网页，验证远程仓库中 main.c 文件更新成功。

```
● Jackson706@localhost:~/work$ git add .
● Jackson706@localhost:~/work$ git commit -m "lab0:完成printf输出"
[main d2c1b7c] lab0:完成printf输出
 1 file changed, 1 insertion(+), 1 deletion(-)
● Jackson706@localhost:~/work$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 320 bytes | 320.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:9704-L/work.git
 ae6bea4..d2c1b7c  main -> main
```

2.任务 4：分支管理、制造并解决合并冲突

创建 feature 分支：`git checkout -b feature`

在 feature 分支中修改 main.c 的 printf 语句，保存文件，提交：`git add main.c`

`git commit -m "feature 分支：修改输出文字"`

切换回主分支 main：`git checkout main`

在 main 分支修改同一行 printf 代码，保存并提交：`git add main.c`

git commit -m "main 分支：修改输出文字"

执行合并，将 feature 合并入 main，触发冲突：git merge feature

终端提示自动合并失败，文件出现冲突标记<<<<<< HEAD、=====、>>>>>> feature。

```
1 file changed, 1 insertion(+), 1 deletion(-)
✖ Jackson706@localhost:~/work$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
```

手动解决冲突：删除所有冲突分隔标记，保留一行自定义输出语句：

printf("Merge finished!\n");

保存文件，标记冲突已解决并提交、推送：

git add main.c

git commit -m "解决冲突，合并 feature 分支到 main"

git push

四、阅读分析

任务 1: 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

答：我此前没有多人协同代码开发的经历。如果进行多人协作开发，一般采用分支开发模式：开发者各自新建独立功能分支，在分支上完成开发与自测；功能完成后再发起合并，经过检查后合并到稳定主分支。这样可以避免多人直接修改 `main` 分支，代码互相覆盖丢失。

思考一下，Git 为什么要设计“暂存-提交”两个步骤？

答：暂存区相当于一个临时缓冲区。如果修改了很多文件，我们可以选择性把部分改动加入暂存，灵活组织提交内容，不用一次性把所有改动全部提交。方便后续代码回溯、查看变更历史，让版本记录更加清晰。

git branch 和 git branch -a 的区别是什么？

答：

git branch: 只列出本地分支，也就是当前电脑上创建的分支，看不到远程服务器上的分支。

git branch -a: 参数-a 代表 all，会同时列出本地分支 + 全部远程分支

任务 3: 在下面的三个网页中任选其二进行阅读，简要概括其内容，并谈谈你对“为什么要学习 Git”这个问题的理解。

(1) Commit Message 规范

Commit Message 规范是一套约定好的提交备注书写格式。它要求提交信息清晰说明本次改动是新增功能、修复 bug、优化代码还是修改文档。规范的提交信息，让团队所有人都能快速看懂每一次提交做了什么，便于定位问题、查阅历史，是团队协作的基础规范。

(2) Git Flow 分支控制

Git Flow 是经典的分支管理模型。它把仓库划分成不同职责的分支：`main` 保存稳定上线代码，`develop` 作为开发主干，`feature` 分支用来开发新功能，`release` 分支用于版本测试，`hotfix` 分支用来紧急修复线上 bug。这套流程保证主分支代码始终稳定，适合大型长期项目的多人协作。

为什么要学习 Git？

答：Git 是目前最主流的分布式版本控制系统。它可以完整记录每一次文件修改，随时回退到历史版本，防止代码丢失。同时它支持多分支并行开发，多人可以互不干扰地同时工作。不管是课程作业、个人项目还是企业软件开发，都依赖 Git 管理代码，学会 Git 是编程开发的必备基础能力。

五、实验总结与体会

本次实验学习了 Git 从基础提交、分支创建到冲突解决的完整流程。一开始我遇到了文件未保存导致 Git 检测不到修改、分支重复创建等小问题，慢慢理解了分支之间相互独立的特点。合并冲突让我直观感受到：当两个分支修改文件同一处代码时，Git 无法自动判断保留哪一份，需要开发者手动决定最终代码。这次实验让我体会到版本控制工具在代码管理中的价值，为以后多人协作开发打下基础。